

# Guía de Supervivencia Examen de Scala

---

 Guía de Supervivencia — Examen de Scala

✦ ÍNDICE

1. Cheat Sheet de Sintaxis Básica (Búsqueda Rápida)

2. Resolución de Ejercicios

Mini Proyecto 3.1 — Clasificador de resultados de un torneo de Twenty-One

Mini Proyecto 3.2 — Analizador de calificaciones de un grupo

# Guía de Supervivencia — Examen de Scala

---

Documento pensado para buscar con `Ctrl + F`. Cada bloque de código tiene comentarios en lenguaje coloquial para encontrar rápido lo que necesitas.

---

## ✈ ÍNDICE

---

1. Cheat Sheet de Sintaxis Básica
  2. Ejercicios Parte 2 (15 ejercicios - JupyterLab)
  3. Mini Proyecto Parte 3.1 — Torneo Twenty-One
  4. Mini Proyecto Parte 3.2 — Analizador de Notas
-

# 1. Cheat Sheet de Sintaxis Básica (Búsqueda Rápida)

## ◆ Variables: `val` vs `var`

```
// cómo declarar una variable que NO se puede cambiar (constante, immutable)
val nombre: String = "Marta"

// cómo declarar una variable que SÍ se puede cambiar (mutable)
var puntuacion: Int = 10

// cómo cambiar el valor de una variable mutable
puntuacion = puntuacion + 5 // ahora vale 15

// esto da ERROR de compilación (reassignment to val):
// nombre = "Otro nombre" // <-- NO se puede hacer, "nombre" es val

// cómo declarar una variable dejando que Scala adivine (infiera) el tipo
val edad = 25 // Scala infiere que es Int
val activo = true // Scala infiere que es Boolean
val pi = 3.14 // Scala infiere que es Double
```

## ◆ Tipos básicos (cómo saber qué tipo usar)

```
val entero: Int = 10 // números enteros
val decimalDoble: Double = 3.14159 // decimales con más precisión (el más usado por defecto)
val decimalFloat: Float = 3.14159f // decimales con menos precisión (nota la "f" al final)
val esVerdad: Boolean = true // true / false
val texto: String = "Hola Scala" // cadenas de texto
val caracter: Char = 'A' // un solo carácter (comillas simples)
```

## ◆ Cómo mostrar cosas por pantalla (print)

```
// cómo imprimir texto normal
println("Hola Scala")

// cómo imprimir concatenando variables (interpolación de strings) - LA FORMA MÁS USADA
val nombre = "Scala"
val version = "2.12.21"
println(s"Hola desde $nombre $version")

// cómo meter una expresión completa dentro de un string interpolado
println(s"El resultado es ${2 + 2}")
```

## ◆ Condicionales `if` / `else if` / `else`

```
// cómo hacer un if-else básico
val puntuacion = 25

if (puntuacion > 21) {
  println("Te has pasado (BUST)")
} else {
  println("Mano válida")
}

// cómo hacer varias condiciones encadenadas (if / else if / else)
val nota = 7

if (nota >= 9) {
  println("EXCELENTE")
} else if (nota >= 7) {
  println("NOTABLE")
} else if (nota >= 5) {
  println("APROBADO")
} else {
  println("SUSPENSO")
}

// cómo usar el if como expresión que devuelve un valor (muy típico en Scala)
val mensaje = if (puntuacion > 21) "BUST" else "VALIDA"
```

## ◆ Operadores relacionales y lógicos (cómo comparar cosas)

```
// operadores relacionales (comparación) - devuelven Boolean
5 > 3      // mayor que      -> true
5 < 3      // menor que      -> false
5 >= 5     // mayor o igual  -> true
5 <= 4     // menor o igual  -> false
5 == 5     // igual a       -> true
5 != 5     // distinto de   -> false

// operadores lógicos - cómo combinar condiciones
true && false // Y lógico (AND) -> false, las DOS tienen que ser true
true || false // O lógico (OR)  -> true, con que UNA sea true vale
!true        // NO lógico (NOT) -> false, invierte el valor

// ejemplo combinando todo (cómo saber si alguien no se ha pasado de 21)
val hand = 18
val noSePasa = !(hand > 21) // true
```

## ◆ Funciones (cómo crear y usar funciones)

```
// cómo definir una función básica que devuelve un Boolean
def bust(puntuacion: Int): Boolean = {
  puntuacion > 21
}

// cómo definir una función que devuelve un String
def saludar(nombre: String): String = {
  s"Hola, $nombre"
}

// cómo definir una función en una sola línea (sin llaves, forma corta)
def esPar(n: Int): Boolean = n % 2 == 0

// cómo definir una función que recibe dos parámetros y devuelve el mayor
def maximo(a: Int, b: Int): Int = {
  if (a > b) a else b
}

// cómo llamar (invocar) a una función
val resultado = bust(24) // true
println(resultado)

// cómo definir una función que no devuelve nada útil (Unit) - solo imprime
def mostrarMensaje(msg: String): Unit = {
  println(msg)
}
```

## ◆ Arrays (cómo trabajar con Array, colección MUTABLE de tamaño fijo)

```

// cómo crear un array con valores directamente
val jugadores = Array("Alex", "Chen", "Marta")

// cómo crear un array vacío/con ceros indicando el tamaño y el tipo
val puntuaciones = new Array[Int](4) // Array(0, 0, 0, 0) - Int se inicializa a 0

// cómo acceder a un elemento de un array por posición (empieza en 0)
val primero = jugadores(0) // "Alex"

// cómo modificar (mutar) un elemento de un array (Sí se puede aunque sea val)
jugadores(0) = "Sindhu" // ahora es Array("Sindhu", "Chen", "Marta")

// esto da ERROR de tipos (no puedes meter un Int en un Array[String]):
// jugadores(0) = 500 // <-- ERROR: type mismatch

// cómo saber cuántos elementos tiene un array (longitud/tamaño)
val tamano = jugadores.length

// cómo recorrer un array con un bucle while
var i = 0
while (i < jugadores.length) {
    println(jugadores(i))
    i = i + 1
}

// cómo recorrer un array con foreach (forma funcional)
jugadores.foreach(jugador => println(jugador))
// forma abreviada (placeholder _)
jugadores.foreach(println)

```

## ◆ Listas (cómo trabajar con List, colección INMUTABLE)

```

// cómo crear una lista
val jugadores = List("Alex", "Chen", "Marta")

// cómo crear una lista vacía
val listaVacía = Nil
// o también:
val otraListaVacía = List[Int]()

// cómo añadir un elemento AL PRINCIPIO de una lista (operador ::, "cons")
val nuevaLista = "Sindhu" :: jugadores
// jugadores SIGUE IGUAL ("Alex", "Chen", "Marta") porque las listas son inmutables
// nuevaLista es ("Sindhu", "Alex", "Chen", "Marta")

// cómo construir una lista desde cero usando Nil y ::
val lista = "Ana" :: "Luis" :: "Marta" :: Nil
// se lee de derecha a izquierda: Nil -> "Marta"::Nil -> "Luis"::("Marta"::Nil) -> ...

// cómo concatenar (unir) dos listas (operador :::)
val listaA = List("Ana", "Luis")
val listaB = List("Pedro", "Sofia")
val listaCompleta = listaA ::: listaB // List("Ana", "Luis", "Pedro", "Sofia")
// listaA y listaB NO cambian, se crea una lista NUEVA

// cómo saber la longitud de una lista
jugadores.length

// cómo invertir una lista
jugadores.reverse

// cómo recorrer una lista con while (necesitas un índice, más típico de Array)
// cómo recorrer una lista con foreach (LO NORMAL en listas)
jugadores.foreach(j => println(j))

```

## ◆ Bucles while y foreach (diferencias)

```
// cómo hacer un bucle while (estilo IMPERATIVO)
// necesita: una variable mutable (var) como contador + incrementarla a mano
var i = 0
val datos = Array(17, 24, 21)
while (i < datos.length) {
    println(datos(i))
    i += 1 // si te olvidas de esto, el bucle es INFINITO
}

// cómo hacer un foreach (estilo FUNCIONAL)
// no necesita contador ni variable mutable, recorre solo
datos.foreach(dato => println(dato))
```

## ◆ Pattern Matching (cómo hacer un `match` tipo switch)

```
// cómo hacer un match básico (equivalente al switch de otros lenguajes)
val dia = 3

val nombreDia = dia match {
    case 1 => "Lunes"
    case 2 => "Martes"
    case 3 => "Miércoles"
    case _ => "Día no válido" // el "_" es el caso por defecto (default)
}

// cómo usar match para clasificar una nota (parecido a un if/else if encadenado)
def clasificacion(nota: Int): String = nota match {
    case n if n >= 9 => "EXCELENTE"
    case n if n >= 7 => "NOTABLE"
    case n if n >= 5 => "APROBADO"
    case _ => "SUSPENSO"
}
```

## ◆ Mutabilidad vs Inmutabilidad (resumen para el examen)

```
// IMMUTABLE: no se puede reasignar
val x = 10
// x = 20 // ERROR

// MUTABLE: sí se puede reasignar
var y = 10
y = 20 // OK

// OJO: un Array declarado con "val" es inmutable la REFERENCIA,
// pero SÍ puedes cambiar sus elementos internos (el array en sí es mutable)
val arr = Array(1, 2, 3)
arr(0) = 99 // OK, esto funciona

// una List declarada con "val" es totalmente inmutable,
// ni la referencia ni el contenido se pueden modificar
val lista = List(1, 2, 3)
// lista(0) = 99 // ERROR: List no tiene este método, no se puede mutar
```

## ◆ Efectos secundarios (side effects) — estilo funcional vs imperativo

```
// ESTILO IMPERATIVO: modifica una variable externa (efecto secundario) - EVITAR si se puede
var total = 0
def sumarAlTotal(valor: Int): Unit = {
    total = total + valor // modifica algo FUERA de la función = efecto secundario
}

// ESTILO FUNCIONAL: no modifica nada externo, solo devuelve un resultado nuevo
def sumar(a: Int, b: Int): Int = {
    a + b // no toca variables externas, siempre devuelve lo mismo con los mismos datos
}
```

## 2. Resolución de Ejercicios

### Ejercicio 1 — Variables, tipos e inferencia

**Enunciado:** > Crea variables que representen la información de una partida de cartas: Nombre del jugador, Número de partidas jugadas, Puntuación actual, Porcentaje de victorias, Indicación de si el jugador continúa activo. Realiza dos versiones: Versión A (declara explícitamente el tipo de cada variable) y Versión B (declara las mismas variables permitiendo que Scala infiera los tipos). Finalmente, muestra los valores por pantalla.

**Código en Scala:**

```
// ----- Versión A: con tipo explícito -----
val nombreA: String = "Marta"
val partidasA: Int = 15
val puntuacionA: Double = 87.5
val porcentajeVictoriasA: Double = 60.0
val activoA: Boolean = true

// ----- Versión B: con inferencia de tipos -----
val nombreB = "Marta"
val partidasB = 15
val puntuacionB = 87.5
val porcentajeVictoriasB = 60.0
val activoB = true

// Mostrar los valores por pantalla
println(s"Versión A -> Nombre: $nombreA, Partidas: $partidasA, Puntuación: $puntuacionA, % Victorias: $porcentajeVictoriasA, Activo: $activoA")
println(s"Versión B -> Nombre: $nombreB, Partidas: $partidasB, Puntuación: $puntuacionB, % Victorias: $porcentajeVictoriasB, Activo: $activoB")
```

**Explicación “Para Tontos”:** - `val nombreA: String = "Marta"` — el `: String` después del nombre de la variable es la **anotación de tipo explícita**: le decimos a Scala “esto es un String, no lo adivines”. - En la Versión B quitamos el `: Tipo` y Scala lo **infiere solo** mirando el valor que le damos ( `"Marta"` → String, `15` → Int, `87.5` → Double, `true` → Boolean). - El resultado final es **exactamente el mismo tipo** en ambas versiones; la única diferencia es si lo escribes tú a mano o deja que el compilador lo deduzca. - `partidas`, `puntuacion` y `porcentaje` deberían poder ser `var` si van a cambiar durante la partida; `nombre` normalmente es `val` porque no cambia. - `s"..."` es un **string interpolado**: la `s` delante de las comillas permite meter variables dentro con `$variable`.

### Ejercicio 2 — `val`, `var` y reasignación

**Enunciado:** > Crea `val jugador = "Marta"` y `var puntuacion = 10`. Incrementa la puntuación en 5 puntos, vuelve a incrementarla en 3 puntos, muestra el resultado final e intenta reasignar el valor de `jugador` (debe producir un error).

**Código en Scala:**

```
val jugador = "Marta"
var puntuacion = 10

puntuacion = puntuacion + 5 // puntuacion = 15
puntuacion = puntuacion + 3 // puntuacion = 18

println(s"Puntuación final de $jugador: $puntuacion")

// Esto genera un ERROR de compilación a propósito (reassignment to val):
jugador = "Sonia"
```

**Explicación “Para Tontos”:** - `puntuacion` es `var`, así que se puede **reasignar** (cambiar su valor) tantas veces como queramos: primero pasa de 10 a 15, luego de 15 a 18. - `jugador` es `val`, así que Scala **no permite** volver a asignarle un valor. Al compilar la última línea, el error que da es del tipo: `error: reassignment to val`. - Esa celda con el error **no se borra**, se deja documentada porque el ejercicio pide demostrar que ese error ocurre. - Regla mental para el examen: **`val` = constante (una sola asignación), `var` = variable (se puede reasignar)**.

### Ejercicio 3 — Tipos numéricos y precisión

**Enunciado:** > Declara un mismo valor decimal utilizando `Double` y `Float` con un número de muchos decimales (ej. `3.14159265358979323846264338327`). Muestra ambos valores. Después crea una variable `Int`, una `Boolean` y una `String`.

## Código en Scala:

```
val piDouble: Double = 3.14159265358979323846264338327
val piFloat: Float = 3.14159265358979323846264338327f

println(s"Double: $piDouble")
println(s"Float: $piFloat")

val numeroEstudiantes: Int = 30
val haAprobado: Boolean = true
val curso: String = "Programación en Scala"

println(s"Estudiantes: $numeroEstudiantes, Aprobado: $haAprobado, Curso: $curso")
```

**Explicación “Para Tontos”:** - `Double` guarda **más dígitos decimales de precisión** (64 bits) que `Float` (32 bits), así que al imprimirlo, `piDouble` mostrará más decimales fieles al número original que `piFloat`. - Para declarar un literal como `Float` hay que poner una **f al final del número** (3.14f); si no la pones, Scala asume que es `Double` por defecto. - Para el número de estudiantes de una clase usarías `Int` (no puede haber medio estudiante, son números enteros). - Para si un estudiante ha aprobado o no usarías `Boolean` (`true`/`false`), porque es una respuesta de sí o no.

## Ejercicio 4 — Función para determinar si una mano se pasa de 21

**Enunciado:** > Define una función `bust` que reciba la puntuación de una mano como `Int` y devuelva un `Boolean`: `true` si la puntuación es mayor que 21, `false` en caso contrario. Pruébala con 18, 21, 22, 30.

### Código en Scala:

```
def bust(puntuacion: Int): Boolean = {
  puntuacion > 21
}

println(bust(18)) // false
println(bust(21)) // false
println(bust(22)) // true
println(bust(30)) // true
```

**Explicación “Para Tontos”:** - `def bust(puntuacion: Int): Boolean = { ... }` — así se define una función en Scala: `def` + nombre + parámetros entre paréntesis con su tipo + `: TipoQueDevuelve` + el cuerpo entre llaves. - Dentro de la función, la última línea (`puntuacion > 21`) es lo que la función **devuelve automáticamente**; en Scala no hace falta escribir `return`. - `puntuacion > 21` es una **expresión booleana**: compara el número con 21 y da como resultado `true` o `false`. - La función **no modifica nada fuera de sí misma**, solo calcula y devuelve un resultado según lo que recibe — eso es programación funcional “pura”.

## Ejercicio 5 — Comparación de dos manos

**Enunciado:** > Define una función `maxHand` que reciba dos valores enteros y devuelva el mayor de los dos utilizando `if` y `else`. Pruébala con 17 y 19, 20 y 18, 21 y 21. Explica qué sucede cuando ambas manos tienen el mismo valor.

### Código en Scala:

```
def maxHand(handA: Int, handB: Int): Int = {
  if (handA > handB) {
    handA
  } else {
    handB
  }
}

println(maxHand(17, 19)) // 19
println(maxHand(20, 18)) // 20
println(maxHand(21, 21)) // 21 (empate, se devuelve handB porque no es "mayor que")
```

**Explicación “Para Tontos”:** - La función compara `handA > handB`. Si es verdadero devuelve `handA`, si no, devuelve `handB`. - Cuando **ambas manos son iguales** (21 y 21), la condición `handA > handB` es `false` (21 no es mayor que 21), así que entra en el `else` y devuelve `handB`. El resultado es correcto en valor (21) aunque técnicamente “elige” el segundo

parámetro por casualidad de cómo está escrita la condición. - No hace falta modificarla porque el enunciado solo pide devolver **el valor mayor**, y en caso de empate ambos valores son iguales, así que da igual cuál se devuelva — el resultado numérico es el mismo.

---

## Ejercicio 6 — Decidir el ganador de una partida

**Enunciado:** > Utiliza la función `bust`. Crea una función `ganador` que reciba `handA: Int` y `handB: Int` y aplique las reglas: si ambas manos superan 21, devuelve 0; si solo `handA` supera 21, devuelve `handB`; si solo `handB` supera 21, devuelve `handA`; si ninguna supera 21, devuelve la mano con mayor puntuación.

**Código en Scala:**

```
def ganador(handA: Int, handB: Int): Int = {
  if (bust(handA) && bust(handB)) {
    0
  } else if (bust(handA)) {
    handB
  } else if (bust(handB)) {
    handA
  } else {
    maxHand(handA, handB)
  }
}

println(ganador(26, 20)) // 20 -> handA se pasa
println(ganador(18, 22)) // 18 -> handB se pasa
println(ganador(24, 25)) // 0 -> ambas se pasan
println(ganador(17, 19)) // 19 -> ninguna se pasa, gana la mayor
println(ganador(21, 20)) // 21 -> ninguna se pasa, gana la mayor
```

**Explicación “Para Tontos”:** - Se reutilizan las funciones `bust` (ejercicio 4) y `maxHand` (ejercicio 5); en Scala, una función puede **llamar a otra función** que ya esté definida. - El `if / else if / else` se evalúa **de arriba a abajo** y se para en la primera condición verdadera: 1. ¿Las dos se pasan de 21? → empate feo, devuelve 0. 2. Si no, ¿se pasa solo `handA`? → gana `handB` automáticamente. 3. Si no, ¿se pasa solo `handB`? → gana `handA` automáticamente. 4. Si no se cumple ninguna de las anteriores, es que **ninguna se pasó**, así que se usa `maxHand` para ver cuál es mayor. - Para `ganador(24, 25)`: `bust(24)` es `true` y `bust(25)` es `true` → entra en el primer `if` → devuelve 0.

---

## Ejercicio 7 — Arrays y mutabilidad

**Enunciado:** > Crea `val jugadores = Array("Alex", "Chen", "Marta")`. Muestra el array original, sustituye "Alex" por "Sindhu", muestra nuevamente el array e intenta asignar el número 500 a una de sus posiciones.

**Código en Scala:**

```
val jugadores = Array("Alex", "Chen", "Marta")
println(jugadores.mkString(", ")) // Alex, Chen, Marta

jugadores(0) = "Sindhu"
println(jugadores.mkString(", ")) // Sindhu, Chen, Marta

// Esto genera un ERROR de tipos a propósito (type mismatch):
jugadores(0) = 500
```

**Explicación “Para Tontos”:** - `jugadores` está declarado con `val`, pero un `Array` es una estructura **mutable por dentro**: `val` solo impide reasignar la variable entera (`jugadores = otroArray`), pero **sí** permite cambiar elementos concretos con `jugadores(indice) = nuevoValor`. - `jugadores(0) = "Sindhu"` cambia solo la posición 0, el resto del array queda igual. - `jugadores(0) = 500` falla porque el array se creó como `Array[String]` (array de textos); Scala es de **tipado fuerte** y no deja meter un `Int` donde espera un `String`. El error es `type mismatch: found Int, required String`. - Diferencia clave: **reasignar la variable** (`jugadores = Array(...)`) sí estaría prohibido por ser `val`; **modificar un elemento** (`jugadores(0) = ...`) está permitido porque el array en sí es mutable. - `mkString(", ")` es un método útil para imprimir un array como texto separado por comas en vez de ver algo como `[Ljava.lang.String;@...`.

---

## Ejercicio 8 — Creación e inicialización de Arrays

**Enunciado:** > Crea un array para almacenar las puntuaciones de cuatro jugadores utilizando `new Array[Int]`. Muestra el array recién creado, asigna manualmente una puntuación a cada posición, muestra el array completo y muestra su longitud con `.length`. Usa las puntuaciones 17, 24, 21, 19.

**Código en Scala:**

```
val puntuaciones = new Array[Int](4)
println(puntuaciones.mkString(", ")) // 0, 0, 0, 0 (valor por defecto)

puntuaciones(0) = 17
puntuaciones(1) = 24
puntuaciones(2) = 21
puntuaciones(3) = 19

println(puntuaciones.mkString(", ")) // 17, 24, 21, 19
println(s"Longitud: ${puntuaciones.length}") // 4
```

**Explicación “Para Tontos”:** - `new Array[Int](4)` crea un array con **espacio reservado para 4 enteros**, pero todavía sin datos “reales” nuestros. - **Pregunta del enunciado:** ¿qué contiene un `Array[Int]` recién creado? → Scala rellena automáticamente con el **valor por defecto del tipo**: para `Int` es `0` (para `Double` sería `0.0`, para `Boolean` sería `false`, para `String` sería `null`). - Luego se asigna manualmente cada posición con `puntuaciones(indice) = valor`, igual que en el ejercicio 7. - `.length` es una propiedad (no necesita paréntesis en su uso normal) que devuelve cuántos elementos tiene el array.

## Ejercicio 9 — Recorrer un Array con `while`

**Enunciado:** > Utiliza `val manos = Array(17, 24, 21, 19, 26)`. Recórrelo con un bucle `while`, mostrando el valor de la mano y el resultado de aplicar `bust`. Usa `var i = 0` y `manos.length`.

**Código en Scala:**

```
val manos = Array(17, 24, 21, 19, 26)

var i = 0
while (i < manos.length) {
  println(s"Mano: ${manos(i)} -> Bust: ${bust(manos(i))}")
  i = i + 1
}
```

**Explicación “Para Tontos”:** - `var i = 0` crea el **contador** que va a ir recorriendo las posiciones del array, empezando en la posición 0. - La condición del `while` (`i < manos.length`) dice “sigue mientras el contador sea menor que el número de elementos” — así nunca te sales del array. - Dentro del bucle: `manos(i)` accede al elemento en la posición actual, y se lo pasamos a `bust(...)` para saber si esa mano se pasa de 21. - `i = i + 1` **incrementa el contador en cada vuelta**; es la parte más importante y la que más se olvida. - El contador **tiene que ser `var`** porque necesita cambiar de valor en cada iteración; si fuera `val` no se podría reasignar y daría error de compilación. - Si no incrementaras `i`, la condición `i < manos.length` sería **siempre verdadera** (porque `i` nunca cambia) y el bucle se ejecutaría **infinitamente**, colgando el programa. - Esto es **estilo imperativo**: usamos una variable mutable (`i`) y vamos dando instrucciones paso a paso de cómo recorrer la colección.

## Ejercicio 10 — Listas e inmutabilidad

**Enunciado:** > Crea `val jugadores = List("Alex", "Chen", "Marta")`. Muestra la lista original, crea una nueva lista añadiendo `"Sindhu"` al principio mediante `::`, muestra ambas listas, comprueba que la original no ha cambiado, y muestra su longitud y la lista invertida con `reverse`.

**Código en Scala:**

```
val jugadores = List("Alex", "Chen", "Marta")
println(jugadores) // List(Alex, Chen, Marta)

val nuevaLista = "Sindhu" :: jugadores
println(nuevaLista) // List(Sindhu, Alex, Chen, Marta)

println(jugadores) // List(Alex, Chen, Marta) -> NO ha cambiado

println(s"Longitud: ${jugadores.length}")
println(s"Invertida: ${jugadores.reverse}")
```

**Explicación “Para Tontos”:** - `::` (se pronuncia “cons”) añade un elemento **al principio** de una lista, pero en vez de modificar la lista original, **crea una lista completamente nueva**. - Por eso, después de hacer `"Sindhu" :: jugadores`, al volver a imprimir `jugadores` sigue teniendo los mismos 3 elementos de siempre: `List` es **inmutable de verdad** (a diferencia del `Array`, aquí ni siquiera se pueden cambiar los elementos internos). - Esto pasa porque en Scala las listas están construidas como una cadena de nodos enlazados; añadir algo “al principio” es barato (solo se crea un nodo nuevo que apunta a la lista vieja), pero la lista vieja **nunca se toca**. - `.reverse` crea (de nuevo, sin modificar la original) una lista con el orden invertido.

## Ejercicio 11 — Construcción y concatenación de listas

**Enunciado:** > Construye una lista utilizando `Nil` y el operador `::` con “Ana”, “Luis”, “Marta”. Crea una segunda lista con “Pedro”, “Sofía”. Concatena ambas con `:::`. Muestra primera lista, segunda lista y lista resultante, demostrando que las originales no cambian.

**Código en Scala:**

```
val primeraLista = "Ana" :: "Luis" :: "Marta" :: Nil
val segundaLista = "Pedro" :: "Sofia" :: Nil

val listaResultante = primeraLista ::: segundaLista

println(s"Primera lista: $primeraLista")      // List(Ana, Luis, Marta)
println(s"Segunda lista: $segundaLista")      // List(Pedro, Sofia)
println(s"Lista resultante: $listaResultante") // List(Ana, Luis, Marta, Pedro, Sofia)
```

**Explicación “Para Tontos”:** - `Nil` representa la **lista vacía**; es el punto de partida obligatorio para construir una lista “a mano” con `::`. - `"Ana" :: "Luis" :: "Marta" :: Nil` se lee de **derecha a izquierda**: primero está `Nil` (lista vacía), luego `"Marta" :: Nil` (lista con un elemento), luego `"Luis" :: (esa lista)`, y finalmente `"Ana" :: (esa lista)`. El resultado final es `List(Ana, Luis, Marta)`. - `:::` (tres dos puntos) sirve para **concatenar dos listas completas** una detrás de otra, creando una lista nueva. - Igual que en el ejercicio 10, ni `primeraLista` ni `segundaLista` se modifican: `:::` genera una **tercera lista independiente** (`listaResultante`), dejando las dos originales intactas — otra prueba de que las listas son inmutables.

## Ejercicio 12 — Operadores relacionales y lógicos

**Enunciado:** > Considera `handA = 18`, `handB = 21`, `handC = 25`. Crea expresiones que respondan: ¿es `handA` mayor que `handB`? ¿es `handB` igual a 21? ¿es `handC` diferente de 21? ¿son `handA` y `handB` menores o iguales a 21? ¿alguna de `handA` o `handC` supera 21? ¿no se ha pasado `handB` de 21?

**Código en Scala:**

```
val handA = 18
val handB = 21
val handC = 25

val pregunta1 = handA > handB           // false
val pregunta2 = handB == 21             // true
val pregunta3 = handC != 21             // true
val pregunta4 = (handA <= 21) && (handB <= 21) // true
val pregunta5 = (handA > 21) || (handC > 21) // true
val pregunta6 = !(handB > 21)           // true

println(pregunta1)
println(pregunta2)
println(pregunta3)
println(pregunta4)
println(pregunta5)
println(pregunta6)
```

**Explicación “Para Tontos”:** - `handA > handB` → `18 > 21` → `false`. - `handB == 21` → compara **igualdad de valor** (con doble `==`, no confundir con un solo `=` que es de asignación) → `true`. - `handC != 21` → `!=` es “distinto de” → `25 != 21` → `true`. - `(handA <= 21) && (handB <= 21)` → las dos condiciones son `true`, y con `&&` (AND) **las dos tienen que cumplirse** → `true`. - `(handA > 21) || (handC > 21)` → con `||` (OR) basta con que **una** sea verdadera; `handA > 21` es `false` pero `handC > 21` es `true` → el resultado total es `true`. - `!(handB > 21)` → primero se calcula `handB > 21` (`false`), y el `!` delante **invierte** ese resultado → `true`.

| Expresión | Resultado | Explicación |
|-----------|-----------|-------------|
|-----------|-----------|-------------|

|                            |       |                                        |
|----------------------------|-------|----------------------------------------|
| handA > handB              | false | 18 no es mayor que 21                  |
| handB == 21                | true  | 21 es igual a 21                       |
| handC != 21                | true  | 25 es distinto de 21                   |
| handA <= 21 && handB <= 21 | true  | ambas son ≤ 21                         |
| handA > 21    handC > 21   | true  | handC sí supera 21                     |
| !(handB > 21)              | true  | handB no se pasa, se invierte el false |

## Ejercicio 13 — `foreach` y funciones como valores

**Enunciado:** > Utiliza `val manos = Array(17, 24, 21, 26, 18)` y la función `bust`. Recorre el array con `foreach`. Haz dos versiones: A) con `while`, B) con `foreach` pasando una función.

**Código en Scala:**

```
val manos = Array(17, 24, 21, 26, 18)

// ----- Versión A: con while -----
var i = 0
while (i < manos.length) {
  println(s"${manos(i)} -> Bust: ${bust(manos(i))}")
  i += 1
}

// ----- Versión B: con foreach -----
manos.foreach(mano => println(s"$mano -> Bust: ${bust(mano)}"))
```

**Explicación “Para Tontos”:** - La **Versión A** necesita un **contador** (`i`) declarado como `var` para saber en qué posición va, y hay que acordarse de incrementarlo (`i += 1`) manualmente. - La **Versión B** con `foreach` **no necesita contador ni variable mutable**: recibe una función `mano => println(...)` (una función anónima, “para cada `mano` de la lista, haz esto”) y Scala se encarga solo de recorrer todos los elementos. - `foreach` es un **método de orden superior**: es una función que recibe **otra función** como parámetro y la aplica a cada elemento de la colección. - Comparación pedida en el enunciado: - Necesita contador → **Versión A (while)**. - Necesita variable `var` para recorrer → **Versión A (while)**. - Se aproxima más al estilo funcional → **Versión B (foreach)**, porque no usa mutabilidad ni pasos manuales, solo declara “qué hacer con cada elemento”.

## Ejercicio 14 — Efectos secundarios y estilo de programación

**Enunciado:** > Analiza `var total = 0` y `def sumarAlTotal(valor: Int) = { total = total + valor }`. Ejecuta la función varias veces, muestra `total` tras cada llamada, explica por qué modifica una variable externa e identifica el efecto secundario. Después crea `def sumar(a: Int, b: Int): Int` que no modifique variables externas.

**Código en Scala:**

```
var total = 0

def sumarAlTotal(valor: Int): Unit = {
  total = total + valor
}

sumarAlTotal(10)
println(s"Total tras la 1ª llamada: $total") // 10

sumarAlTotal(5)
println(s"Total tras la 2ª llamada: $total") // 15

sumarAlTotal(20)
println(s"Total tras la 3ª llamada: $total") // 35

// ----- Versión sin efectos secundarios -----
def sumar(a: Int, b: Int): Int = {
  a + b
}

val resultado = sumar(10, 5)
println(s"Resultado de sumar(10, 5): $resultado") // 15, y "total" no se ve afectado
```

**Explicación “Para Tontos”:** - `sumarAlTotal` **no recibe** `total` **como parámetro**, pero lo modifica igualmente porque `total` es una variable declarada **fuera** de la función (en un ámbito más amplio) y la función puede “ver” y tocar variables externas si son `var`. - Cada vez que se llama a `sumarAlTotal`, el valor de `total` **cambia permanentemente**, y si llamas a la función varias veces con el mismo valor, el resultado va cambiando (no es predecible solo mirando el parámetro). - **El efecto secundario** es precisamente eso: la función, además de (o en lugar de) devolver un resultado, **modifica el estado de algo que existe fuera de ella** (`total`). - `sumar`, en cambio, solo usa `a` y `b` (lo que recibe como parámetros) y **siempre devuelve el mismo resultado** para las mismas entradas, sin tocar nada externo — es una función **pura**.

| Característica                  | <code>sumarAlTotal</code> | <code>sumar</code>      |
|---------------------------------|---------------------------|-------------------------|
| Modifica datos externos         | Sí ( <code>total</code> ) | No                      |
| Devuelve un resultado calculado | No ( <code>Unit</code> )  | Sí ( <code>Int</code> ) |
| Utiliza efecto secundario       | Sí                        | No                      |
| Estilo predominante             | Imperativo                | Funcional               |

## Ejercicio 15 — Programa integrado: torneo de Twenty-One

**Enunciado:** > Combina `val jugadores = List("Alex", "Chen", "Marta", "Sindhu")` y `val manos = Array(18, 24, 21, 20)`. Usa `bust`, recorre las puntuaciones mostrando si cada mano se ha pasado, determina la mejor puntuación válida, usa `if/else if/else`, operadores relacionales/lógicos, al menos una función propia y al menos un `while` o `foreach`. Añade una segunda ronda con `Array(22, 19, 20, 21)` sin modificar las listas originales.

**Código en Scala:**

```
val jugadores = List("Alex", "Chen", "Marta", "Sindhu")
val manos = Array(18, 24, 21, 20)
val manosRonda2 = Array(22, 19, 20, 21)

// función propia que evalúa el estado de una mano
def estadoMano(puntuacion: Int): String = {
  if (bust(puntuacion)) "BUST" else "VALIDA"
}

// función propia que calcula la mejor puntuación válida de una ronda
def mejorPuntuacionValida(puntuaciones: Array[Int]): Int = {
  var mejor = 0
  var i = 0
  while (i < puntuaciones.length) {
    if (!bust(puntuaciones(i)) && puntuaciones(i) > mejor) {
      mejor = puntuaciones(i)
    }
    i += 1
  }
  mejor
}

// ----- Ronda 1: recorrido con while -----
println("--- Ronda 1 ---")
var i = 0
while (i < manos.length) {
  println(s"${jugadores(i)} -> ${manos(i)} -> ${estadoMano(manos(i))}")
  i += 1
}
println(s"Mejor puntuación válida ronda 1: ${mejorPuntuacionValida(manos)}")

// ----- Ronda 2: recorrido con foreach (usando índices de jugadores) -----
println("--- Ronda 2 ---")
manosRonda2.zipWithIndex.foreach { case (puntuacion, indice) =>
  println(s"${jugadores(indice)} -> $puntuacion -> ${estadoMano(puntuacion)}")
}
println(s"Mejor puntuación válida ronda 2: ${mejorPuntuacionValida(manosRonda2)}")
```

**Explicación “Para Tontos”:** - `estadoMano` reutiliza `bust` (ejercicio 4) y devuelve un `String` en lugar de un `Boolean` — así el `println` queda más legible (“BUST”/“VALIDA”). - `mejorPuntuacionValida` recorre el array con un **while con contador** `var i` (estilo imperativo) y va guardando en `mejor` (también `var`, porque necesita actualizarse) la puntuación más alta que **no** se haya pasado de 21. - La condición `!bust(puntuaciones(i)) && puntuaciones(i) > mejor` combina un operador lógico (`&&`) con uno relacional (`>`) y una negación (`!`): “si esta mano NO se pasa de 21 Y además es mayor que la mejor que llevamos hasta ahora, actualízala”. - En la Ronda 2 se usa `foreach` (estilo más funcional) junto con `zipWithIndex`, que empareja cada puntuación con su posición (índice) para poder acceder al nombre del jugador correspondiente en la lista

jugadores, sin necesitar un contador manual. - Ni jugadores, ni manos, ni manosRonda2 se modifican en ningún momento: solo se **leen** sus valores. - Resumen para el informe final: - **Mutable:** el contador `i` y la variable `mejor` dentro de `mejorPuntuacionValida`. - **Inmutable:** jugadores, manos, manosRonda2 (nunca se reasignan ni se les cambian elementos). - **Funciones:** estadoMano, mejorPuntuacionValida, y se reutiliza `bust`. - **Estructuras de control:** `if/else` dentro de `estadoMano` y dentro del `while`. - **Más funcional:** el recorrido de la Ronda 2 con `foreach`. - **Más imperativo:** el recorrido de la Ronda 1 con `while` y el uso de variables mutables en `mejorPuntuacionValida`.

---

## Mini Proyecto 3.1 — Clasificador de resultados de un torneo de Twenty-One

**Enunciado:** > Programa que analice los resultados de varias manos de un torneo de Twenty-One. Cada jugador tiene un nombre y una puntuación. El programa debe determinar si cada jugador se ha pasado de 21, qué jugadores tienen puntuación válida, cuál es la mejor puntuación válida de la ronda, cuántos jugadores se han pasado y cuántos siguen en partida. Datos: `jugadores = List("Alex", "Chen", "Marta", "Sindhu", "Luis")`, `puntuaciones = Array(18, 24, 21, 20, 26)`. Segunda ronda: `Array(22, 19, 20, 21, 17)`.

**Código en Scala (Main.scala):**

```
object Main extends App {

    val jugadores = List("Alex", "Chen", "Marta", "Sindhu", "Luis")
    val puntuaciones = Array(18, 24, 21, 20, 26)
    val puntuacionesRonda2 = Array(22, 19, 20, 21, 17)

    // Función que dice si una puntuación se pasa de 21
    def bust(puntuacion: Int): Boolean = {
        puntuacion > 21
    }

    // Función que devuelve el estado como texto
    def estadoMano(puntuacion: Int): String = {
        if (bust(puntuacion)) "BUST" else "VALIDA"
    }

    // Función que decide la mejor mano entre dos puntuaciones
    def mejorMano(handA: Int, handB: Int): Int = {
        if (bust(handA) && bust(handB)) {
            0
        } else if (bust(handA)) {
            handB
        } else if (bust(handB)) {
            handA
        } else if (handA > handB) {
            handA
        } else {
            handB
        }
    }

    // Función que procesa una ronda completa y devuelve estadísticas
    def procesarRonda(nombres: List[String], puntuacionesRonda: Array[Int], titulo: String): Int = {
        println(s"--- $titulo ---")

        var i = 0
        var validas = 0
        var busts = 0
        var mejor = 0

        while (i < puntuacionesRonda.length) {
            val puntuacion = puntuacionesRonda(i)
            val estado = estadoMano(puntuacion)
            println(s"${nombres(i)} -> $puntuacion -> $estado")

            if (bust(puntuacion)) {
                busts += 1
            } else {
                validas += 1
                if (puntuacion > mejor) {
                    mejor = puntuacion
                }
            }
            i += 1
        }

        println("--- Resumen de la ronda ---")
        println(s"Jugadores: ${nombres.length}")
        println(s"Manos válidas: $validas")
        println(s"Bust: $busts")
        println(s"Mejor puntuación válida: $mejor")

        mejor
    }
}
```

```

// Ronda 1
val mejorRonda1 = procesarRonda(jugadores, puntuaciones, "Resumen de la ronda 1")

// Ronda 2 (misma lista de jugadores, no se modifica)
val mejorRonda2 = procesarRonda(jugadores, puntuacionesRonda2, "Resumen de la ronda 2")

// Comparación de rondas
println("--- Comparación final ---")
println(s"Mejor puntuación ronda 1: $mejorRonda1")
println(s"Mejor puntuación ronda 2: $mejorRonda2")

if (mejorRonda1 > mejorRonda2) {
    println("La ronda 1 tuvo la mejor puntuación")
} else if (mejorRonda2 > mejorRonda1) {
    println("La ronda 2 tuvo la mejor puntuación")
} else {
    println("Ambas rondas tuvieron la misma mejor puntuación")
}

// Versión adicional con foreach (comparativa while vs foreach)
println("--- Estado de la ronda 1 usando foreach ---")
puntuaciones.foreach(p => println(s"$p -> ${estadoMano(p)}"))
}

```

**Explicación “Para Tontos”:** - El programa se organiza en **varias funciones pequeñas** (`bust`, `estadoMano`, `mejorMano`, `procesarRonda`) en vez de meter todo el código en una sola pieza — esto es justo lo que pide el enunciado (“dividir el problema en varias funciones”). - `procesarRonda` es la función central: recibe la lista de nombres, el array de puntuaciones y un texto para el título, y hace **todo el trabajo repetitivo** (recorrer, clasificar, contar y calcular la mejor puntuación) usando un `while` con contador `i`. - Dentro del `while`: por cada posición se mira si la puntuación es `bust` o no; si **no** es `bust`, se suma a `validas` y se comprueba si es mayor que la `mejor` puntuación vista hasta ahora (patrón típico de “buscar el máximo” recorriendo un array). - Se llama a `procesarRonda` **dos veces** (una por cada ronda) pasando `jugadores` sin tocarlo nunca — así se cumple el requisito de “no modificar la lista original”. - La comparación final usa un `if/else` `if/else` para decidir cuál ronda tuvo mejor puntuación, o si hubo empate. - Al final se añade una versión con `foreach` que hace lo mismo que el bucle `while` pero de forma más funcional, para poder comparar ambos estilos en el `README.md` tal y como pide el enunciado (el `while` necesita contador y variable mutable; el `foreach` no).

---

## Mini Proyecto 3.2 — Analizador de calificaciones de un grupo

**Enunciado:** > Aplicación que analice las calificaciones de un grupo de estudiantes: almacenar nombres y notas, determinar quién aprueba y quién suspende, calcular estadísticas básicas, identificar la mejor nota y mostrar un resumen final. Datos: `estudiantes = List("Ana", "Luis", "Marta", "Pedro", "Sofia")`, `notas = Array(8,4,10,6,3)`. Segunda evaluación: `Array(9,5,8,7,6)`.

**Código en Scala (Main.scala):**

```
object Main extends App {

    val estudiantes = List("Ana", "Luis", "Marta", "Pedro", "Sofia")
    val notas = Array(8, 4, 10, 6, 3)
    val notasSegundaEvaluacion = Array(9, 5, 8, 7, 6)

    // Función que determina si una nota está aprobada
    def aprobado(nota: Int): Boolean = {
        nota >= 5
    }

    // Función que devuelve el estado como texto
    def estadoNota(nota: Int): String = {
        if (aprobado(nota)) "APROBADO" else "SUSPENSO"
    }

    // Función que devuelve la mayor de dos notas
    def maxNota(a: Int, b: Int): Int = {
        if (a > b) a else b
    }

    // Función que clasifica una nota en un rango
    def clasificacion(nota: Int): String = {
        if (nota >= 9) {
            "EXCELENTE"
        } else if (nota >= 7) {
            "NOTABLE"
        } else if (nota >= 5) {
            "APROBADO"
        } else {
            "SUSPENSO"
        }
    }

    // Función que procesa una evaluación completa
    def procesarEvaluacion(nombres: List[String], notasEvaluacion: Array[Int], titulo: String): (Int, Int) = {
        println(s"--- $titulo ---")

        var i = 0
        var aprobados = 0
        var suspensos = 0
        var mejorNota = 0

        while (i < notasEvaluacion.length) {
            val nota = notasEvaluacion(i)
            println(s"${nombres(i)} -> $nota -> ${clasificacion(nota)}")

            if (aprobado(nota)) {
                aprobados += 1
            } else {
                suspensos += 1
            }

            mejorNota = maxNota(mejorNota, nota)
            i += 1
        }

        println("--- Resumen del grupo ---")
        println(s"Estudiantes: ${nombres.length}")
        println(s"Aprobados: $aprobados")
        println(s"Suspensos: $suspensos")
        println(s"Mejor nota: $mejorNota")

        (mejorNota, aprobados)
    }
}
```

```

// Primera evaluación
val (mejorNota1, aprobados1) = procesarEvaluacion(estudiantes, notas, "Primera evaluación")

// Segunda evaluación (misma lista de estudiantes, no se modifica)
val (mejorNota2, aprobados2) = procesarEvaluacion(estudiantes, notasSegundaEvaluacion, "Segunda evaluación")

// Comparación de evaluaciones
println("--- Comparación de evaluaciones ---")
println(s"Mejor nota primera evaluación: $mejorNota1")
println(s"Mejor nota segunda evaluación: $mejorNota2")
println(s"Aprobados primera evaluación: $aprobados1")
println(s"Aprobados segunda evaluación: $aprobados2")

if (aprobados2 > aprobados1) {
    println("El grupo ha mejorado")
} else if (aprobados2 < aprobados1) {
    println("El grupo ha empeorado")
} else {
    println("El grupo se ha mantenido igual")
}

// Nueva lista añadiendo un estudiante al principio (sin modificar la original)
val nuevosEstudiantes = "Carlos" :: estudiantes
println(s"Lista original: $estudiantes")
println(s"Lista nueva: $nuevosEstudiantes")
}

```

**Explicación “Para Tontos”:** - La estructura es prácticamente un **calco del mini proyecto 3.1**, pero aplicada a notas en vez de manos de cartas — buena señal de que dominando un patrón (recorrer + clasificar + contar + comparar) se resuelven varios ejercicios parecidos. - `aprobado(nota)` usa el operador relacional `>=` para saber si la nota llega al aprobado (5 o más). - `clasificacion(nota)` es un `if/else if/else` **en cascada**: primero pregunta por la condición más restrictiva (`>= 9`) y va bajando; si pusieras las condiciones en el orden contrario, todas las notas caerían en el primer `if` que sea verdadero por error de lógica, así que **el orden importa**. - `procesarEvaluacion` devuelve una **tupla** (`Int, Int`) — es decir, dos valores a la vez (la mejor nota y el número de aprobados) — para poder usarlos después en la comparación final entre evaluaciones. Se “desempaqueta” con `val (mejorNota1, aprobados1) = ...`. - La comparación final entre evaluaciones se hace mirando el **número de aprobados** (tal y como pide el enunciado) con un `if/else if/else` que decide si el grupo mejoró, empeoró o se mantuvo igual. - `"Carlos" :: estudiantes` crea una lista nueva con Carlos añadido al principio, sin tocar `estudiantes` — la razón es la misma que en el ejercicio 10: `::` siempre genera una lista nueva porque `List` es inmutable.